ReX, Activation–Deactivation, and SMT-based Explanation

1 Using AD in an SMT pipeline

1. Restricting interpolant support

Using ADfor computing a candidate set for Capture strategy. The focused explanation can be shown by minimum norm point with the rest of pixels occluded/dimed.

Here, there is no novelty,instead, it only made the explanations more useful and readable by giving more focused explanations automatically. It could be done manually by asking the user to provide an interesting part of the image.

2. AD-seeded guess-and-check

The plan is to let ReX guess the answer and let the solver check it. Running ReX with ConvAD takes seconds and gives a candidate set of pixels X_0. Instead of searching for an explanation with the solver, we ask the solver one question: is $\varphi_{X_0} \wedge \psi$ unsatisfiable?

- UNSAT — there is no way to change the free pixels and lose the class, so X_0 *is* an abductive explanation. We got it in one query instead of d.
- SAT — the solver found an input that keeps X_0 at the sample values but gets a different class. So X_0 is too small. We add something to it and ask again.

The rest of this subsection is about that second case, which is where the real work is.

The solver returns a full assignment x^*, which is a complete image. The class can change because many free pixels each moved a little, with no single one responsible.

What we get directly is the set of free pixels that the solver actually moved away from the sample:

$$D \;=\; \{\, i \notin X_0 \;:\; x^*_i \neq s_i \,\}.$$

Everything else the solver left alone, so it cannot have contributed to this particular counterexample.

1. D **is never empty.** If it were, then x^* would agree with s on the free pixels and on X_0, so x^* would just be s — and s has class c, which contradicts the solver having found a different class. So you can never get stuck with nothing to add.
2. **Adding any one pixel of D rules out this counterexample.** If you put $i \in D$ into the set, the next query requires $x_i = s_i$, but $x^*_i \neq s_i$. So x^* can no longer come back as an answer. Any choice makes progress.
3. **The loop always finishes.** Each round adds at least one pixel, and there are only d pixels, so there are at most d rounds. That is the same worst case as VeriX, so this can only be faster, never slower.

Note: We can make D smaller by preferring the original values of the free pixels in the above query.

Choosing what to add

The useful question is :

Which of the changed pixels do I have to put back to their sample values before the class becomes c again?

Call such a set $H \subseteq D$. We want a small one:

$$\kappa\big(x^*[H \mapsto s_H]\big) = c,$$

where $x^*[H \mapsto s_H]$ means "x^*, but with the pixels in H reset to their sample values".

There is always at least one answer, namely $H = D$: putting back every pixel the solver moved gives you s itself, which has class c. So we can start from D and try to make it smaller by dropping pixels one at a time and keeping each drop only if the class still comes out right:

```
BLAME(x*, s, D):
    H = D
    for i in D:
        if class( x*[H \ {i} -> s] ) == c:
            H = H \ {i}                # i was not needed, drop it
    return H
```

The important thing here is the cost. Every test inside BLAME is one forward pass through the network, not a solver call.

The expensive solver calls stay in the outer loop only. The whole loop is then:

```
X = X0                                 # AD-seeded candidate
repeat:
    (sat, x*) = CHECK(phi_X and psi)
    if unsat:  return X                # done, verified explanation
    D = { i not in X : x*_i != s_i }
    H = BLAME(x*, s, D)
    X = X + H
```

AD is usefull in:

- **The order in BLAME.** Which minimal H you end up with depends on the order you try dropping pixels. Ordering by AD responsibility means the pixels AD thinks are useless get dropped first, so what remains is concentrated on what the model really uses.
- **The class test inside BLAME.** Every test asks "what happens if these pixels are not there". It's cheap and precise enough for a heuristic.

Two ways to make it faster (Claude's opinion)

- **Ask for several counterexamples at once.** Most solvers can return more than one model per call. Compute an H for each, then add the pixel that appears in the most of them. This shrinks the number of rounds a lot and costs no extra solver calls.
- **Grow the counterexample before blaming it.** One point is thin evidence. If you first widen x^* into a box of inputs that all get the wrong class, then pixels that can vary freely inside that box are clearly not the cause, and pixels pinned near one value are. This costs more per round but gives much better refinement — and since interpolation is already in the pipeline, it is closer to hand here than it would be elsewhere.

Two warnings (Claude's opinion)

- **It matters how free the free pixels are.** If they range over the whole domain, the solver can return wild images, D will be large, and the rounds will be slow. If they are restricted to a small neighbourhood of the sample, counterexamples stay close to s, D is small, and blame is much more meaningful. This loop should work far better in the distance-restricted setting, and that is where I would test it first.

- **The loop itself is not new, and that is good.** What it computes is a set that "hits" all the correction sets of all the counterexamples, and the link between explanations and counterexamples of this kind is already known (Ignatiev, Narodytska and Marques-Silva, *On Relating Explanations and Adversarial Examples*, NeurIPS 2019 — worth checking the exact reference). That means you inherit existing results instead of having to prove convergence yourself, and the part that is actually yours is narrower and easier to defend: using AD to seed X_0 and to answer the class test inside BLAME.

3. Pruning the encoding

Under a committed mask, ConvAD makes an identifiable set of neurons identically zero. Those can be dropped from $\psi_{\mathcal{M}}$ outright: no ITE term, no ReLU case split. Since ReLU splits are the source of exponential blow-up, this is potentially the largest saving of the three.

The caveat is essential: Thm. 4.1 guarantees $N = N'$ *only* on the all-ones mask (no masking); under any real mask they differ by construction. Pruning therefore proves a property of N', not N. Two ways to stay sound: use N' for *search* only and confirm the final candidate on N (i.e. the loop above); or state the result about N' deliberately — defensible on the AD paper's own terms, since it argues N' is the more faithful object, but a formal-methods audience will reasonably object.

The only case it can be useful, is when we focus on some part of the image, and realy don't care about the rest, then we can prune the network in the same way. Which is in fact explicit in the encoding any ways.

Obstacles

- **ConvAD is CNN-only.**
- **Granularity.** ReX reasons over boxes, SMT over individual features. Not purely an obstacle — the box hierarchy suits coarse-to-fine verification.

The papers

- **ReX** — Chockler, Kelly, Kroening, Sun, *Causal Explanations for Image Classifiers*, JAIR 86 (2026), arXiv:2411.08875. Defines the notion of explanation and gives a black-box algorithm computing it.
- **Taxonomy** — Kelly, Chockler, *Sufficient, Necessary and Complete Causal Explanations*, arXiv:2507.23497. Splits that notion into a family; see Section 1.
- **AD** — Chanchal, Kelly, Chockler, *Activation–Deactivation*, arXiv:2510.01038v2. The subject of most of what follows.

Also referenced: VeriX (Wu, Wu, Barrett, arXiv:2212.01051); the FM 2026 survey *Formally Explaining Neural Network Classification*; and *Space Explanations*, arXiv:2511.22498.